

Digitalna forenzika

Forenzika fajl sistema

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Anja Tonsa Milovanović , br. ind. 1815 | Mentor: Prof. dr Bratislav Predić

Elektronski fakultet, Niš | jul 2026.

Digitalna forenzika

Predstavlja primenu naučnih i tehničkih principa za prikupljanje, analizu i prezentovanje digitalnih dokaza sa pravnom relevantnošću.

Osnovni principi:

Integritet podataka

Originalni podaci se ne diraju – radi se na forenzičkim kopijama

Dokumentovanje

Svaki korak mora biti detaljno zabeležen sa vremenskim oznakama

Ponovljivost

Analiza mora dati iste rezultate uz iste podatke i metode

Pravna prihvatljivost

Dokazi moraju biti validni i u skladu sa proceduralnim standardima

Digitalna forenzika

Predstavlja primenu naučnih i tehničkih principa za prikupljanje, analizu i prezentovanje digitalnih dokaza sa pravnom relevantnošću.

Proces forenzičke analize:

- Prikupljanje podataka – Identifikacija i zaplena digitalnih dokaza, izrada forenzičkih kopija i očuvanje integriteta originalnih podataka.
- Ispitivanje podataka – Analiza podataka i metapodataka, uključujući oporavak obrisanih podataka, logova i drugih digitalnih tragova.
- Analiza podataka – Korišćenje forenzičkih metoda i alata za otkrivanje skrivenih informacija i povezivanje nalaza sa potencijalnim akterima.
- Izveštavanje – Izrada formalnog izveštaja sa rezultatima istrage, zaključcima i preporukama za dalje postupanje.

Osnove fajl sistema

Osnovne funkcije fajl sistema

- Organizacija podataka – Hijerarhijska struktura direktorijuma i fajlova omogućava jednostavnu navigaciju i pristup podacima.
- Upravljanje prostorom – Dodela, praćenje i oslobađanje prostora na disku pomoću tabela alokacije i bitmapa.
- Kontrola pristupa – Definisanje dozvola za čitanje, izmenu i izvršavanje fajlova radi zaštite podataka.
- Upravljanje metapodacima – Čuvanje informacija o fajlovima (veličina, vlasnik, vremenske oznake, dozvole) i očuvanje integriteta podataka.
- Efikasno pronalaženje podataka – Korišćenje indeksnih struktura i algoritama za brz pristup fajlovima.

Osnove fajl sistema

Komponente fajl sistema



Fajl – Osnovna jedinica za čuvanje podataka koja predstavlja logičku celinu (dokument, slika, program).



Direktorijum – Struktura koja organizuje fajlove i omogućava lakšu navigaciju kroz sistem.



Metapodaci – Informacije koje opisuju osobine fajla, kao što su veličina, vlasnik, dozvole i vremenske oznake.



Kontrolni blok fajla – Interna struktura koja čuva metapodatke, lokaciju podataka na disku i informacije potrebne za upravljanje fajlom.

Osnove fajl sistema

Organizacija podataka na disku

■ Fajl sistem povezuje logičku strukturu fajlova i direktorijuma sa fizičkim rasporedom podataka na disku.

■ Sektor je osnovna fizička jedinica skladištenja, dok klasteri predstavljaju osnovnu jedinicu alokacije prostora u fajl sistemu.

■ Fajl sistem raspoređuje fajlove kroz jedan ili više klastera na memorijskom uređaju.

■ Neiskorišćeni prostor u poslednjem klasteru fajla koji nastaje kada veličina fajla ne popuni ceo dodeljeni klaster predstavlja slack space.

Fajl (3000 B)

Slack Space (1096 B)

← Klaster 4096 B →

Najpoznatiji fajl sistemi

FAT / exFAT

USB, SD kartice

Jednostavni i široko kompatibilni fajl sistemi zasnovani na tabeli alokacije, najčešće korišćeni na USB memorijama i prenosivim uređajima, ali bez naprednih bezbednosnih funkcija.

NTFS

Windows (NT+)

Savremeni Windows fajl sistem zasnovan na MFT strukturi, koji pruža napredne funkcije kao što su dozvole pristupa, enkripcija, journaling i bogate forenzičke artefakte.

HFS / APFS

macOS, iOS

Apple fajl sistemi namenjeni macOS uređajima, pri čemu APFS donosi modernu organizaciju prostora, snapshot-e i enkripciju, dok je HFS značajan uglavnom za starije sisteme.

Ext4

Linux

Standardni Linux fajl sistem koji koristi extents i journaling za visoke performanse, pouzdanost i očuvanje integriteta podataka.

Tehnike skrivanja podataka u fajl sistemu

Hidden atribut

Najjednostavniji. Fajlovi se sakriju kroz Windows podešavanja. Lako se otkrije uključivanjem prikaza skrivenih fajlova.

Steganografija na nivou fajl sistema

Specijalni fajl sistemi (npr. StegFS) distribuuju podatke kao slučajni šum. Bez ključa nemoguće razlikovati od praznog prostora.

Nealocirani prostor

Brisanje = oslobađanje prostora. Podaci fizički ostaju. File carving pronalazi fajlove po zaglavljima (npr. FF D8 FF za JPEG).

Alternate Data Streams

NTFS specifičnost. Dodatni "tokovi" fajla, nevidljivi u Exploreru. Može čak sadržati i izvršni kod.

Slack Space

Direktan upis iza logičkog kraja fajla. Struktura fajla se ne menja. Nevidljivo standardnim alatima.

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

CLI alat za skrivanje podataka u slack space-u Ext4 fajl sistema

Python

jezik implementacije

Ext4

podržani fajl sistem

4

osnovne komande

SAT

tabela za evidenciju

- Osnovna ideja je da alat može da omogući smeštanje fajlova u slack prostor se bez izmene strukture fajl sistema.
- Analizom postojećih fajlova pronalazi dostupan slack postor i deli ga na slack sektore (prodrzumevana veličina 512B).
- Alat omogućava skrivanje, čitanje i brisanje fajlova.
- Radi evidencije o skrivenim podacima implementirana je SAT(Slack Allocation Table) kao JSON fajl koji čuva evidenciju o tome koji su fajlovi skriveni i gde se nalaze.

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Okruženje korišćenja alata i priprema uređaja

- Alat je namenjen prvenstveno za rad sa USB memorijskim uređajima, čime se izbegava rizik od oštećenja podataka na sistemskom disku.
- Pre upotrebe potrebno je pripremiti USB uređaj: kreirati particiju, obrisati postojeći sadržaj i formatirati je u ext4 fajl sistem.
- Nakon povezivanja uređaj dobija device putanju (npr. `/dev/sdb1`) i mount putanju (npr. `/mnt/testusb`).
- Alat koristi obe putanje: device putanju za direktan pristup sektorima i slack prostoru, a mount putanju za analizu fajlova i pronalaženje raspoloživog slack prostora.

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Funkcionalnosti alata

hide

Skrivanje jednog ili više fajlova u slack prostor

read

Čitanje i rekonstrukcija skrivenih fajlova

delete

Brisanje podataka (prepisivanje nulama)

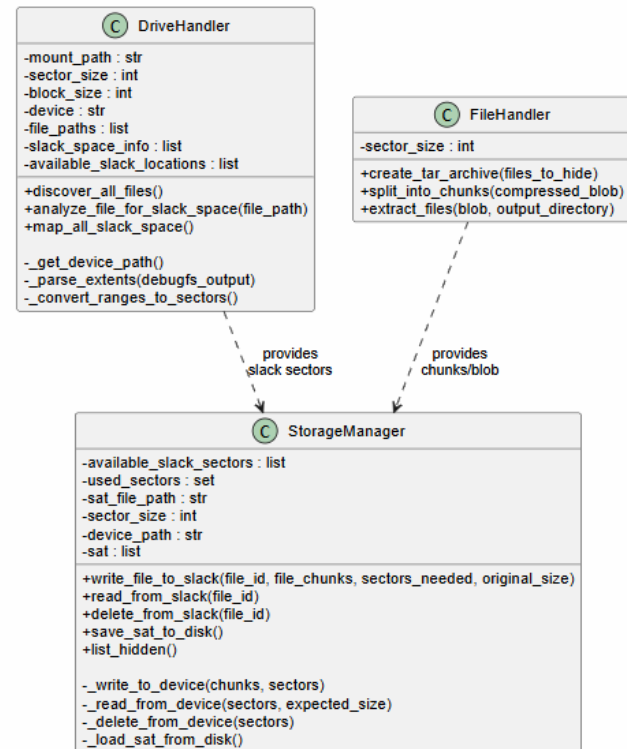
list_hidden_files

Prikaz liste skrivenih fajlova iz SAT tabele

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Arhitektura sistema

- Arhitektura sistema organizovana je kroz tri osnovne klase DriveHandler, FileHandler i StorageManager.
- Funkcije read(), hide(), list_hidden_files(), delete() pozivaju metode ovih klasa i na taj način omogućavaju funkcionalnosti sistema.
- Na slici prikazan je UML klasni dijagram implementiranog rešenja.



Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Arhitektura sistema

DriveHandler

- Pronalazi sve fajlove na particiji
- Analizira inode + extent zapise (debugfs)
- Izračunava slack prostor svakog fajla
- Mapira slack → fizički sektori (512B)

FileHandler

- Kreira TAR arhivu ulaznih fajlova
- GZIP kompresija podataka
- Deli sadržaj na sektore
- GZIP dekompresija + raspakivanje

StorageManager

- Alocira slobodne slack sektore
- Direktan upis/čitanje (seek + write)
- Brisanje prepisivanjem nulama
- Vodi i čuva SAT tabelu (JSON)

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Pronalaženje slack prostora

- Pronalazak svih fajlova na ciljnoj particiji omogućen je metodom `discover_all_files()` koja koristi `os.walk()`.
- Nakon pronalaska svih fajlova poziva se metoda `map_all_slack_space()` koja za svaki fajl poziva funkciju `analyze_file_for_slack_space()`, čiji je zadatak određivanje raspoloživog slack prostora.
- Vrš se pribavljanje osnovnih metapodataka korišćenjem funkcije `os.stat()`. Na taj način dobijaju se veličina fajla i broj inode-a koji predstavlja jedinstveni identifikator fajla unutar ext4 fajl sistema.

```
def analyze_file_for_slack_space(self, file_path):  
    ...  
    stat_info = os.stat(file_path)  
    file_size = stat_info.st_size  
    inode_number = stat_info.st_ino  
    ...
```

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Pronalaženje slack prostora

- Dobijeni inode broj koristi se za pristup informacijama o fizičkoj organizaciji fajla na disku pomoću alata debugfs. Analizom extent zapisa, koji se dobija izvršavanjem stat komande, moguće je odrediti fizičku lokaciju poslednjeg bloka koji pripada određenom fajlu.

```
def analyze_file_for_slack_space(self, file_path):  
    ...  
    cmd = ['debugfs', '-R', f'stat <{inode_number}>', self.device]  
    result = subprocess.run(cmd, capture_output=True, text=True)  
    extent_info = self._parse_extents(result.stdout)  
    ...
```

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Pronalaženje slack prostora

- Dobijeni izlaz zatim se obrađuje metodom `_parse_extents()`. Tokom obrade izdvajaju se fizičke adrese blokova koje pripadaju fajlu. Pošto se slack prostor nalazi na kraju poslednjeg alociranog bloka, za dalju analizu koristi se poslednji pronađeni extent. Nakon određivanja početnog i završnog bloka vrši se konverzija blokova u bajt adrese. Na osnovu lokacije poslednjeg bloka izračunava se stvarni kraj fajla i količina neiskorišćenog prostora unutar tog bloka. Ovaj prostor predstavlja slack space koji može biti iskorišćen za skladištenje skrivenih podataka.

```
def analyze_file_for_slack_space(self, file_path):  
    ...  
    # converting blocks to byte offsets  
    extent_start_bytes = extent_start_block * self.block_size  
    extent_end_bytes = (extent_end_block + 1) * self.block_size  
  
    # actual ending of a file  
    file_end_bytes = extent_start_bytes + file_size  
  
    slack_start = file_end_bytes  
    slack_end = extent_end_bytes  
    slack_size = slack_end - slack_start  
    ...
```

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Pronalaženje slack prostora

- Dobijeni opsezi slack prostora zatim se prevode u listu fizičkih sektora veličine jednog sektora koji je podrazumevano 512 bajta, metodom `_convert_ranges_to_sectors()`. Početna adresa svakog slack sektora dodaje u listu raspoloživih lokacija za skladištenje podataka `available_slack_locations`.

```
def _convert_ranges_to_sectors(self):
    #converting slack space byte ranges to sector offsets
    print("[Drive handler info]: Converting byte ranges to sector offsets...")

    for slack_info in self.slack_space_info:
        slack_start = slack_info['slack_start']
        slack_end = slack_info['slack_end']

        for sector_offset in range(slack_start, slack_end, self.sector_size):
            if sector_offset < slack_end:
                self.available_slack_locations.append(sector_offset)

    self.available_slack_locations.sort()
    print(f"[Drive handler info]: Mapped {len(self.available_slack_locations)}
    available sectors")
```

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Skrivanje podataka

- Proces skrivanja podataka sastoji se iz nekoliko faza: pripreme podataka, određivanja potrebnog prostora za upis i fizičkog upisa u slack prostor.
- Nakon pronalaženja raspoloživih sektora u slack prostoru, sledeći korak predstavlja priprema podataka za skladištenje. Ovaj deo funkcionalnosti implementiran je u okviru klase `FileHandler`, čiji je zadatak da ulazne fajlove pripremi za upis u prethodno mapirane sektore.
- Korisnik kao ulaz može proslediti pojedinačne fajlove ili listu fajlova koje želi da sakrije. Svaki ulaz se najpre objedinjuje u TAR arhivu, nakon čega se vrši GZIP kompresija kako bi se smanjila količina prostora potrebna za skladištenje. Ovo je omogućeno funkcijom `create_tar_archive(files_to_hide)`.
- U okviru funkcije `split_into_chunks(compressed_blob)` dobijeni kompresovani sadržaj koji predstavlja binarne podatke deli se na segmente veličine jednog slack sektora. Ukoliko poslednji segment ne popunjava ceo sektor, vrši se dopunjavanje nulama kako bi svi segmenti imali jednaku veličinu. Funkcija vraća listu segmenata, kao i broj segmenata koje je potrebno upisati u slack prostor. Informacija o stvarnoj veličini kompresovanog sadržaja čuva se u SAT tabeli i koristi se tokom rekonstrukcije.

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Skrivanje podataka

```
def split_into_chunks(self, compressed_blob):

    print(f"[File handler info]: Splitting blob into {self.sector_size}-byte
    chunks...")
    chunks = []

    # Split blob into slack sector-sized pieces
    for i in range(0, len(compressed_blob), self.sector_size):
        chunk = compressed_blob[i:i + self.sector_size]

        # If the last chunk is smaller than sector_size, pad it with zeros
        if len(chunk) < self.sector_size:
            chunk = chunk + b'\x00' * (self.sector_size - len(chunk))

        chunks.append(chunk)

    print(f"[File handler info]: Created
```

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Skrivanje podataka

- Nakon pripreme podataka, poziva se funkcija `write_to_slack()` klase `StorageManager`, koja određuje broj potrebnih sektora i pronalazi dovoljan broj slobodnih lokacija u prethodno formiranoj mapi slack prostora.

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Skrivanje podataka

```
def write_to_slack(self, blob_id, blob_chunks, sectors_needed, original_size):

    free_sectors = [s for s in self.available_slack_sectors if s not in
self.used_sectors]
    print("Free sectors", free_sectors[0:7])

    if len(free_sectors) < sectors_needed:
        print(f"[Storage manager info]: Not enough slack space. Need
        {sectors_needed}, have {len(free_sectors)}")
        return None

    allocated_sectors = free_sectors[:sectors_needed]
    print("Allocated sectors: ", allocated_sectors)
    first_sector = allocated_sectors[0]
    last_sector = allocated_sectors[-1]
    # write chunks to disk
    try:
        self._write_to_device(blob_chunks, allocated_sectors)
    except Exception as e:
        print(f"[Storage manager error]: Error writing to device: {e}")
```

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Skrivanje podataka

- Ukoliko postoji dovoljan broj slobodnih sektora, podaci se direktno upisuju na uređaj korišćenjem operacija seek() i write() u okviru metode _write_to_device().

```
def _write_to_device(self, chunks, sectors):
    try:
        with open(self.device_path, 'r+b') as device:
            for chunk, sector_offset in zip(chunks, sectors):
                device.seek(sector_offset)
                device.write(chunk)

        print(f"[Storage manager info]: Successfully wrote {len(chunks)} sectors")
    except Exception as e:
        print(f"[Storage manager error]: Error writing to device: {e}")
```

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Skrivanje podataka

- Po završetku upisa kreira se zapis u SAT tabeli koji omogućava kasnije pronalaženje i rekonstrukciju podataka.

```
sat_entry = {  
    'file_id': file_id,  
    'first_sector': first_sector,  
    'last_sector': last_sector,  
    'num_sectors': sectors_needed,  
    'data_size': original_size,  
    'timestamp': datetime.now().isoformat()  
}
```

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Evidencija skrivenih podataka korišćenjem SAT tabele

- Prilikom pokretanja programa SAT tabela se učitava sa diska pozivom `_load_sat_from_disk()` u okviru konstruktora `StorageManager` klase i na osnovu nje se formira skup zauzetih sektora `used_sectors`. Ovim postupkom sprečava se mogućnost da novi podaci budu upisani preko već postojećih skrivenih sadržaja.

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Evidencija skrivenih podataka korišćenjem SAT tabele

```
def _load_sat_from_disk(self):
    sat_table=[]
    try:
        with open(self.sat_file_path, 'r') as f:
            sat_table = json.load(f)

        self.used_sectors = set()
        for entry in sat_table:
            first = entry['first_sector']
            num = entry['num_sectors']
            for i in range(num):
                self.used_sectors.add(first + i*self.sector_size)

        print(f"[Storage manager info]: SAT loaded from {self.sat_file_path}
({len(sat_table)} files)")

    except Exception as e:
        with open(self.sat_file_path, 'w') as f:
            json.dump(sat_table, f)
        print(f"[Storage manager error]: No available SAT, making a new one.")
    return sat_table
```

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Rekonstrukcija podataka

- Proces čitanja skrivenih podataka započinje pronalaženjem odgovarajućeg zapisa u SAT tabeli u okviru metode `read_from_slack(file_id)` na osnovu prosleđivanja identifikatora željenog fajla. Na osnovu podataka o lokaciji sektora određuju se fizičke adrese sa kojih treba izvršiti čitanje.
- Sadržaj svih sektora koji pripadaju izabranom fajlu učitava se sa uređaja i spaja u jedinstveni binarni blok koji se vraća pozivom `_read_from_device(sectors_to_read, data_size)`. Pošto se prilikom skladištenja poslednji sektor dopunjava nulama, višak podataka se uklanja korišćenjem originalne veličine sadržaja sačuvane u SAT tabeli.

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Rekonstrukcija podataka

```
def read_from_slack(self, blob_id):

    # find the SAT entry
    sat_entry = None
    for entry in self.sat:
        if entry['file_id'] == blob_id:
            sat_entry = entry
            break

    if not sat_entry:
        print(f"[Storage manager info]: File '{blob_id}' not found in SAT")
        return None

    #read file from sectors
    first_sector = sat_entry['first_sector']
    last_sector = sat_entry['last_sector']
    data_size = sat_entry['data_size']

    sectors_to_read = [
        sector for sector in self.available_slack_sectors
        if first_sector <= sector <= last_sector
    ]

    file_data = self._read_from_device(sectors_to_read, data_size)

    print(f"[Storage manager info]: Read '{blob_id}' from sectors {first_sector}-{sat_entry['last_sector']}")
    return file_data
```

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Brisanje podataka

- Brisanje skrivenih podataka realizovano je prepisivanjem svih slack sektora koji pripadaju određenom fajlu nulama. Nakon uspešnog brisanja fizičkog sadržaja odgovarajući zapis se uklanja iz SAT tabele.

Implementacija alata za skrivanje podataka u okviru fajl sistema korišćenjem slack space-a

Testiranje

Skrivanje	<pre>sudo python3 slack_tool.py hide -d /dev/sdb1 -f file1.txt file2.pdf \ -m /mnt/testusb -s ./sat.json</pre>
Čitanje	<pre>sudo python3 slack_tool.py read -d /dev/sdb1 -f file1.txt \ -m /mnt/testusb -s ./sat.json -o ./output</pre>
Brisanje	<pre>sudo python3 slack_tool.py delete -d /dev/sdb1 -f file1.txt \ -m /mnt/testusb -s ./sat.json</pre>
Pregled liste skrivenih fajlova	<pre>sudo python3 slack_tool.py list -d /dev/sdb1 -s ./sat.json -m /mnt/testusb</pre>